

Introduction to Programming with Haskell

Guarded definitions, Currying

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Guarded definitions

- Use **guards** to selectively enable definitions

```
xor12 :: Bool → Bool → Bool
xor12 b1 b2
  | b1 == b2      = False
  | b1 /= b2      = True
```

Guarded definitions

- Use **guards** to selectively enable definitions

```
xor12 :: Bool → Bool → Bool
xor12 b1 b2
  | b1 == b2      = False
  | b1 /= b2      = True
```

- Definition has two clauses

Guarded definitions

- Use **guards** to selectively enable definitions

```
xor12 :: Bool → Bool → Bool
xor12 b1 b2
  | b1 == b2      = False
  | b1 /= b2      = True
```

- Definition has two clauses
- Each clause is guarded by a conditional expression

Guarded definitions

- Use **guards** to selectively enable definitions

```
xor12 :: Bool → Bool → Bool
xor12 b1 b2
  | b1 == b2      = False
  | b1 /= b2      = True
```

- Definition has two clauses
- Each clause is guarded by a conditional expression
- Value of the function is the first expression whose guard evaluates to True

Guarded definitions

- Use **guards** to selectively enable definitions

```
xor12 :: Bool → Bool → Bool
xor12 b1 b2
  | b1 == b2      = False
  | b1 /= b2      = True
```

- Definition has two clauses
- Each clause is guarded by a conditional expression
- Value of the function is the first expression whose guard evaluates to True
- **Syntax**: Note the indentation

Guarded definitions

- Can mix pattern matching and guards

```
xor13 :: Bool → Bool → Bool
xor13 False b2      = b2
xor13 True b2
  | b2 == False    = True
  | b2 == True     = False
```

Guarded definitions

- Can mix pattern matching and guards

```
xor13 :: Bool → Bool → Bool
xor13 False b2      = b2
xor13 True b2
  | b2 == False  = True
  | b2 == True   = False
```

- Two definitions

Guarded definitions

- Can mix pattern matching and guards

```
xor13 :: Bool → Bool → Bool
xor13 False b2      = b2
xor13 True b2
  | b2 == False  = True
  | b2 == True   = False
```

- Two definitions
- Second definition is guarded

Guarded definitions

- Guards may overlap

Guarded definitions

- Guards may overlap
- Guards need not be exhaustive

```
xor14 :: Bool → Bool → Bool
xor14 b1 b2
  | b1 == True  = not b2
  | b2 == False = b1
```

Guarded definitions

- Guards may overlap
- Guards need not be exhaustive

```
xor14 :: Bool → Bool → Bool
xor14 b1 b2
  | b1 == True  = not b2
  | b2 == False = b1
```

- For `xor14 True False`, both guards evaluate to **True**

Guarded definitions

- Guards may overlap
- Guards need not be exhaustive

```
xor14 :: Bool → Bool → Bool
xor14 b1 b2
  | b1 == True  = not b2
  | b2 == False = b1
```

- For `xor14 True False`, both guards evaluate to **True**
 - But the result returned is **not False**, since the first guard evaluates to **True**

Guarded definitions

- Guards may overlap
- Guards need not be exhaustive

```
xor14 :: Bool → Bool → Bool
xor14 b1 b2
  | b1 == True  = not b2
  | b2 == False = b1
```

- For `xor14 True False`, both guards evaluate to `True`
 - But the result returned is `not False`, since the first guard evaluates to `True`
- For `xor14 False True`, neither guard evaluates to `True` (runtime error)

```
** Exception: Non-exhaustive patterns in function xor
```

Guarded definitions

- Always provide a catchall guard at the end

Guarded definitions

- Always provide a catchall guard at the end
- Guards need not be exhaustive

```
xor15 :: Bool → Bool → Bool
xor15 b1 b2
  | b1 == True  = not b2
  | otherwise   = b2
```


Guarded definitions

- Always provide a catchall guard at the end
- Guards need not be exhaustive

```
xor15 :: Bool → Bool → Bool
xor15 b1 b2
  | b1 == True  = not b2
  | otherwise   = b2
```

- **otherwise** is defined to be **True**, so the guard always passes

Guarded definitions

- Always provide a catchall guard at the end
- Guards need not be exhaustive

```
xor15 :: Bool → Bool → Bool
xor15 b1 b2
  | b1 == True  = not b2
  | otherwise   = b2
```

- **otherwise** is defined to be **True**, so the guard always passes
- This RHS applies if all the previous guards fail

Using case

- A final way to define `xor`

```
xor16 :: Bool → Bool → Bool
xor16 b1 b2 = case b1 of
    False → b2
    True  → not b2
```

Using case

- A final way to define `xor`

```
xor16 :: Bool → Bool → Bool
xor16 b1 b2 = case b1 of
    False → b2
    True   → not b2
```

- Like `switch/case` in C

Using case

- A final way to define `xor`

```
xor16 :: Bool → Bool → Bool
xor16 b1 b2 = case b1 of
  False → b2
  True   → not b2
```

- Like `switch/case` in C
 - **Warning:** Expression, not a statement!

Using case

- Can have nested cases too!

```
xor17 :: Bool → Bool → Bool
xor17 b1 b2 = case b1 of
  False → case b2 of
    False → False
    True  → True
  True  → not b2
```

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`
- Normally functions come with an **arity**

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`
- Normally functions come with an **arity**
 - Number of arguments

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`
- Normally functions come with an **arity**
 - Number of arguments
- Instead, assume all functions take only one input!

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`
- Normally functions come with an **arity**
 - Number of arguments
- Instead, assume all functions take only one input!
- Return **functions** if necessary!

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`
- Normally functions come with an **arity**
 - Number of arguments
- Instead, assume all functions take only one input!
- Return **functions** if necessary!
- `plus`, applied to `n`, returns a function that adds `n` to any input

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`
- Normally functions come with an **arity**
 - Number of arguments
- Instead, assume all functions take only one input!
- Return **functions** if necessary!
- `plus`, applied to `n`, returns a function that adds `n` to any input
- This is called **currying**, for the logician **Haskell Curry**

Multiple inputs

- Recall that we write `plus n m`, not `plus(n,m)`
- Normally functions come with an **arity**
 - Number of arguments
- Instead, assume all functions take only one input!
- Return **functions** if necessary!
- `plus`, applied to `n`, returns a function that adds `n` to any input
- This is called **currying**, for the logician **Haskell Curry**
- Haskell is also named after Haskell Curry

Multiple inputs

- `plus(n,m) = n+m`

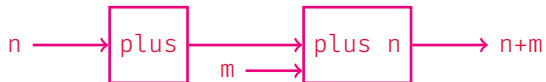


Multiple inputs

- `plus(n,m) = n+m`



- `plus n m = n+m`

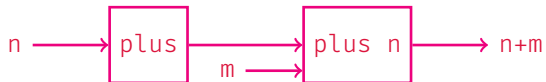


Multiple inputs

- `plus(n,m) = n+m`



- `plus n m = n+m`



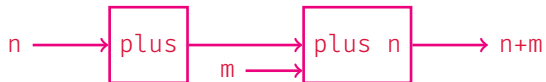
- `plus n` maps `m` to `n+m`

Multiple inputs

- `plus(n,m) = n+m`



- `plus n m = n+m`



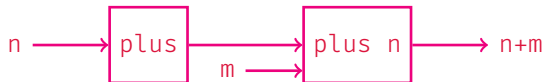
- `plus n` maps `m` to `n+m`
- `plus n :: Int → Int`

Multiple inputs

- `plus(n,m) = n+m`



- `plus n m = n+m`



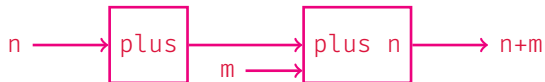
- `plus n` maps `m` to `n+m`
- `plus n :: Int → Int`
- `plus` maps `n` to `plus n`

Multiple inputs

- `plus(n,m) = n+m`



- `plus n m = n+m`



- `plus n` maps `m` to `n+m`
- `plus n :: Int → Int`
- `plus` maps `n` to `plus n`
- `plus :: Int → (Int → Int)`

Multiple inputs

- `plus3 n m l = n+m+l`



Multiple inputs

- `plus3 n m l = n+m+l`



- `plus3 n m` maps `l` to `n+m+l`

Multiple inputs

- `plus3 n m l = n+m+l`



- `plus3 n m` maps `l` to `n+m+l`
- `plus3 n m :: Int → Int`

Multiple inputs

- `plus3 n m l = n+m+l`



- `plus3 n m` maps `l` to `n+m+l`
- `plus3 n m :: Int → Int`
- `plus3 n` maps `m` to `plus3 n m`

Multiple inputs

- `plus3 n m l = n+m+l`



- `plus3 n m` maps `l` to `n+m+l`
- `plus3 n m :: Int → Int`
- `plus3 n` maps `m` to `plus3 n m`
- `plus3 n :: Int → (Int → Int)`

Multiple inputs

- `plus3 n m l = n+m+l`



- `plus3 n m` maps `l` to `n+m+l`
- `plus3 n m :: Int → Int`
- `plus3 n` maps `m` to `plus3 n m`
- `plus3 n :: Int → (Int → Int)`
- `plus3` maps `n` to `plus3 n`

Multiple inputs

- `plus3 n m l = n+m+l`



- `plus3 n m` maps `l` to `n+m+l`
- `plus3 n m :: Int → Int`
- `plus3 n` maps `m` to `plus3 n m`
- `plus3 n :: Int → (Int → Int)`
- `plus3` maps `n` to `plus3 n`
- `plus3 :: Int → (Int → (Int → Int))`

Multiple inputs

- Consider a function with many arguments

```
f x1 x2 ... xn = e
```

Multiple inputs

- Consider a function with many arguments

```
f x1 x2 ... xn = e
```

- Suppose each x_i is of type **Int**, e is of type **Bool**

Multiple inputs

- Consider a function with many arguments

```
f x1 x2 ... xn = e
```

- Suppose each x_i is of type **Int**, e is of type **Bool**
- Then we have:

```
f :: Int → (Int → ( ... (Int→Bool) ... ))
```

Multiple inputs

- Consider a function with many arguments

```
f x1 x2 ... xn = e
```

- Suppose each x_i is of type **Int**, e is of type **Bool**
- Then we have:

```
f :: Int → (Int → ( ... (Int→Bool) ... ))
```

- Correspondingly, we should write

```
( ... ((f x1) x2) ... ) xn = e
```

Multiple inputs

- Consider a function with many arguments

```
f x1 x2 ... xn = e
```

- Suppose each x_i is of type **Int**, e is of type **Bool**
- Then we have:

```
f :: Int → (Int → ( ... (Int→Bool) ... ))
```

- Correspondingly, we should write

```
( ... ((f x1) x2) ... ) xn = e
```

- Too many parentheses!

Multiple inputs

- Fortunately, Haskell makes life easy for us

Multiple inputs

- Fortunately, Haskell makes life easy for us
- Implicit bracketing for types is from the right, so

```
f :: Int → Int → ... → Int → Bool
```

actually means

```
f :: Int → (Int → (... → (Int → Bool) ... ))
```

Multiple inputs

- Likewise, function application brackets from left, so

```
f x1 x2 ... xn
```

actually means

```
( ... ((f x1) x2) ... ) xn
```

Binary functions and infix operators

- Recall the built-in functions `div` and `mod` that behave as follows:

```
div 29 3 = 9
```

```
mod 29 3 = 2
```

Binary functions and infix operators

- Recall the built-in functions `div` and `mod` that behave as follows:

```
div 29 3 = 9
```

```
mod 29 3 = 2
```

- Binary functions can be used as infix operators by enclosing them inside **backticks** – like ``div`` and ``mod``

```
29 `div` 3 = 9
```

```
29 `mod` 3 = 2
```

Binary functions and infix operators

- Recall the built-in functions `div` and `mod` that behave as follows:

```
div 29 3 = 9
```

```
mod 29 3 = 2
```

- Binary functions can be used as infix operators by enclosing them inside **backticks** – like ``div`` and ``mod``

```
29 `div` 3 = 9
```

```
29 `mod` 3 = 2
```

- On the other hand, infix operators can be used in prefix form by enclosing in parentheses

```
(+) 5 3 = 8
```

```
(/) 8 3 = 2.6666666666666665
```

Sectioning

- `(+)` works on two numbers

Sectioning

- `(+)` works on two numbers
- What is `(+) n`?

Sectioning

- $(+)$ works on two numbers
- What is $(+) \ n$?
- It is a function that adds n to any input it receives

Sectioning

- $(+)$ works on two numbers
- What is $(+) \ n$?
- It is a function that adds n to any input it receives
- Special syntax in Haskell: $(n+)$

Sectioning

- $(+)$ works on two numbers
- What is $(+) \ n$?
- It is a function that adds n to any input it receives
- Special syntax in Haskell: $(n+)$
- Fixes the first argument: $(5+) \ 3 = 8$

Sectioning

- $(+)$ works on two numbers
- What is $(+) \ n$?
- It is a function that adds n to any input it receives
- Special syntax in Haskell: $(n+)$
- Fixes the first argument: $(5+) \ 3 = 8$
- $(+n)$ fixes the second argument: $(+5) \ 8 = 13$

Sectioning

- $(+)$ works on two numbers
- What is $(+) \ n$?
- It is a function that adds n to any input it receives
- Special syntax in Haskell: $(n+)$
- Fixes the first argument: $(5+) \ 3 = 8$
- $(+n)$ fixes the second argument: $(+5) \ 8 = 13$
- Expressions like $(+5)$ and $(3+)$ are called **sections**

Sectioning

- Can be applied to other binary operators too

Sectioning

- Can be applied to other binary operators too
- $(\star 8) \ 3 = 24$

Sectioning

- Can be applied to other binary operators too
- $(\star 8) \ 3 = 24$
- $(8 /) \ 3 = 2.6666666666666665$

Sectioning

- Can be applied to other binary operators too
- $(\star 8) \ 3 = 24$
- $(8 /) \ 3 = 2.6666666666666665$
- $(/8) \ 3 = 0.375$

Sectioning

- Can be applied to other binary operators too
- $(\star 8) \ 3 = 24$
- $(8 /) \ 3 = 2.6666666666666665$
- $(/8) \ 3 = 0.375$
- $(8-) \ 3 = 5$

Sectioning

- Can be applied to other binary operators too
- $(\ast 8) \ 3 = 24$
- $(8 /) \ 3 = 2.6666666666666665$
- $(/8) \ 3 = 0.375$
- $(8 -) \ 3 = 5$
- $(- \ 8) \ 3$ does not work, though! Interpreted as a negative number, not a section

Sectioning

- Can be applied to other binary operators too
- $(\star 8) \ 3 = 24$
- $(8 /) \ 3 = 2.6666666666666665$
- $(/8) \ 3 = 0.375$
- $(8 -) \ 3 = 5$
- $(- \ 8) \ 3$ does not work, though! Interpreted as a negative number, not a section
- Use `subtract` instead

Sectioning

- Can be applied to other binary operators too
- $(\star 8) \ 3 = 24$
- $(8 /) \ 3 = 2.6666666666666665$
- $(/8) \ 3 = 0.375$
- $(8 -) \ 3 = 5$
- $(- \ 8) \ 3$ does not work, though! Interpreted as a negative number, not a section
- Use `subtract` instead
- $(\text{subtract } 8) \ 3 = -5$

Recursive definitions

- **Base case:** $f(0)$ is given

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$
- Factorial

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$
- Factorial
 - $0! = 1$

Recursive definitions

- **Base case:** $f(0)$ is given
- **Inductive step:** For $n > 0$, $f(n)$ is defined in terms of f on smaller values: $f(n-1), f(n-2), \dots, f(0)$
- Factorial
 - $0! = 1$
 - For $n > 0$, $n! = n \times (n-1)!$

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- **Integer** represents integers of arbitrarily large magnitude

```
>>> factorial 40
8159152832478977343456112695961158942720000000000
```

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- **Integer** represents integers of arbitrarily large magnitude

```
>>> factorial 40
8159152832478977343456112695961158942720000000000
```

- **Int** has lower and upper bounds

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n      = n * factorial (n-1)
```

- **Integer** represents integers of arbitrarily large magnitude

```
>>> factorial 40
8159152832478977343456112695961158942720000000000
```

- **Int** has lower and upper bounds
- -2^{63} and $2^{63} - 1$ on my machine

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n      = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0    = 1
factorial n    = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?
 - Note the parentheses again!

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n      = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?
 - Note the parentheses again!
 - `factorial -1` is `1` subtracted from the function `factorial`

Example program: Factorial

- In Haskell:

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n      = n * factorial (n-1)
```

- Note the parentheses around `n-1`
 - `factorial n-1` is interpreted as `(factorial n) - 1`
- No guarantee of termination
- What is `factorial (-1)`?
 - Note the parentheses again!
 - `factorial -1` is `1` subtracted from the function `factorial`
 - Type error!

Example program: Factorial

- Fix the factorial function to work on negative values

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n
  | n ≥ 0      = n * factorial (n-1)
  | otherwise = n * factorial (n+1)

>>> factorial (-20)
2432902008176640000
```

Example program: Factorial

- Fix the factorial function to work on negative values

Example program: Factorial

- Fix the factorial function to work on negative values
- Sometimes there is no natural way to extend the definition

```
factorial :: Integer → Integer
factorial 0      = 1
factorial n
  | n ≥ 0      = n * factorial (n-1)
  | otherwise = error "Works only for non-negative inputs"

>>> factorial (-20)
Works only for non-negative inputs
```